FORGE

Autonomous Test Orchestration Agent

Approach & design rationale — why the pipeline is built the way it is, what every stage decides, and the trade-offs made on purpose along the way.

What this deck covers

01 The problem with testing web apps today

02 Our thesis — three pillars

03 The eight-stage pipeline, end to end

04 Explore & Design/Psychology Critic

05 Plan, Critic, and the re-plan loop

06 Generate, Run, and Triage

07 Heal — the scorer and the five vetoes

08 Two decisions we defend on paper (ADR-001, ADR-017)

09 The Report and the live dashboard

10 Why this is better, and where we're honest it isn't finished

Every existing approach fails the same way

Manual QA doesn't scale with release cadence. Scripted E2E suites rot the moment a label changes. And the newest fix — "AI self-healing" — introduces a worse failure mode than the one it solves.

01 **Manual testing doesn't scale.**

A human re-clicking through the app before every release is slow, inconsistent, and the first thing cut under deadline pressure.

02 **Scripted suites are brittle by construction.**

A locator written against today's DOM breaks the moment a class name or button label moves — regardless of whether the feature underneath still works.

03 **Naive "self-healing" hides the exact failures that matter most.**

A healer built to just find the closest matching element can't tell "the button moved" from "the button now deletes the order." Commercial healers ship this today.

04 **The two failures look identical at the moment they happen.**

`0 elements matched` is the symptom either way. Every downstream decision depends on telling them apart — before touching anything.

Heal what's safe. Refuse what isn't. Show the arithmetic.

PILLAR 01

Evidence before arithmetic before verdict

No stage jumps straight to a decision. A failure is diagnosed, then scored, then gated — every step leaves the evidence that produced it in the report.

"0.71 said heal; V2 said no; the veto wins."

PILLAR 02

Deterministic-first, model-optional

Every verdict that can be computed with a formula and a fixed rule is. The model is consulted for judgment, never for permission — it can lower a verdict, never raise one.

"Pulling the API key changes what the report says, not what the pipeline does."

PILLAR 03

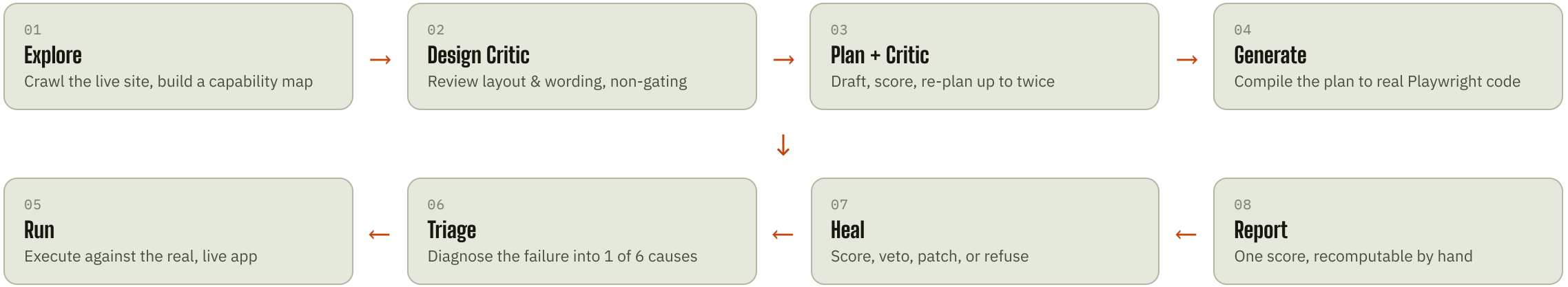
Refuse loudly, never fix quietly

A false "fixed" costs a production incident and the credibility of every future green build. A false "needs review" costs one engineer a minute. The pipeline is built around that asymmetry.

"A suite that heals through real bugs stops carrying information."

One pass, eight stages, in a fixed order

A URL goes in on the left. A scored, evidenced verdict comes out on the right. Every arrow below is a real, named handoff in the code — not a metaphor.



Deterministic-first, model-optional

Only two call sites in the whole pipeline ever touch a model — and neither one can block, unblock, or override a verdict arithmetic already reached.

- **The model tells us what we missed. It does not get to decide whether we ship.**
  A model-authored coverage gap is clamped to `MAJOR` in code before it's even merged — only eight deterministic rules can mint or clear a `BLOCKER`. See ADR-017.
- **The same asymmetry governs healing.**
  The healer's adjudication call can only ever *lower* a verdict, never raise one — a model cannot unblock a fired veto. See ADR-001.
- **This is a testable guarantee, not a policy.**
  Every golden case runs with `ANTHROPIC_API_KEY` unset and produces the identical verdict, five times in a row.
- **Why it matters to a reviewer.**
  A gate that only holds when the network is up is not a gate. Determinism is what turns "the model decided" into "arithmetic decided, and the model's advice is on the record too."

STAGE 01

Explore

EXPLORE

FORGE opens the real, live URL in a real browser and clicks through it the way a first-time visitor would — every page, button, link and form — logging in first if credentials are given. Nothing is tested yet; this stage only builds a map of what exists.

→ **Perception is structural, not visual.**
Pages are read from the accessibility tree — roles and accessible names — not pixels, so the map survives a re-skin.

→ **Duplicate pages collapse to one state.**
`/product/:sku` variants are recognised as the same screen via a state signature, so the crawl doesn't explode on a catalogue.

→ **The crawl always terminates.**
Four halt reasons are all reachable and guaranteed — a state budget, a time budget, or the frontier genuinely running out of new links.

WHAT COMES OUT

States	**deduplicated screens**
Transitions	**ways to move between them**
Affordances	**every clickable / fillable control**
Authenticated	**true / false**
Halt reason	**why the crawl stopped**

STAGE 02

Design & Psychology Critic

DESIGN

A second, independent review runs on the same screens the Explorer already captured — this time asking whether the page is actually usable, not just functional. It is deliberately kept separate from the Coverage Critic (ADR-018): it can inform the report, but it can never gate the pipeline.

→ **Grounded in real research, not vibes.**
Checks map to growth.design's CLEAR framework, Fitts's Law, Miller's Law, and WCAG AA contrast — every finding cites the principle it's based on.

→ **Evidence-layer discipline.**
A finding must originate from structural, semantic, or geometric evidence. A screenshot only corroborates — it never mints a finding on its own.

→ **Additive, non-gating, by design.**
Neither the pipeline's exit code nor its pass/fail outcome changes because of anything this stage finds.

DELIGHT SCORE — 5 COMPONENTS

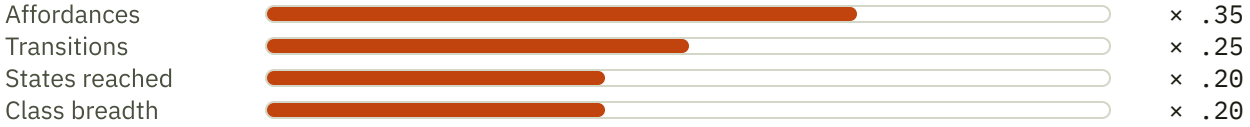
Copywriting	**CLEAR "C"**
Layout	**heading hierarchy, above-the-fold**
Emphasis	**Fitts's Law, contrast**
Accessibility	**WCAG AA, labels**
Reward*	***placeholder — needs before/after capture**

STAGE 03

Plan & the Coverage Critic

CRITIC

A test plan is drafted from the capability map, then scored against everything the Explorer found. Below the floor, it's sent back — at most twice — before FORGE proceeds anyway and names exactly what's still missing.



THE RE-PLAN LOOP

Floor	**0.70**
Max rounds	**2 (TG-6 cap)**

PASS ≥ 0.70

REPLAN below floor, rounds left

ACCEPT_RISK below floor, cap reached

Generate, then Run — for real

An accepted plan is compiled into actual Playwright code, then executed against the real, live application. Nothing here is mocked or replayed.

GENERATE

The compiler turns each scenario's steps into the same clicks, typing and checks a human tester would perform by hand — written down so they run again, unattended, exactly the same way every time.

→ **Locators favor role + accessible name.**
The most robust identity a control has, before falling back to anything more brittle.

RUN

Each scenario gets its own real browser context and executes against production or staging — not a fixture, not a snapshot.

→ **If the button isn't there today, the test finds out today.**
A pass means the flow works right now, not that it worked when the test was written.

Triage — diagnose before you touch anything

A failure is classified into exactly one of six causes before any repair is even considered. A moved button and a genuinely broken feature must never be handled the same way.

LOCATOR_BREAK
The address moved; an identical control still exists elsewhere. A healing candidate.

PRODUCT_BUG
The thing expected genuinely isn't there. Reported, never patched.

CONTENT_DRIFT
Real copy or price changed — a claim to update by hand, not to auto-heal.

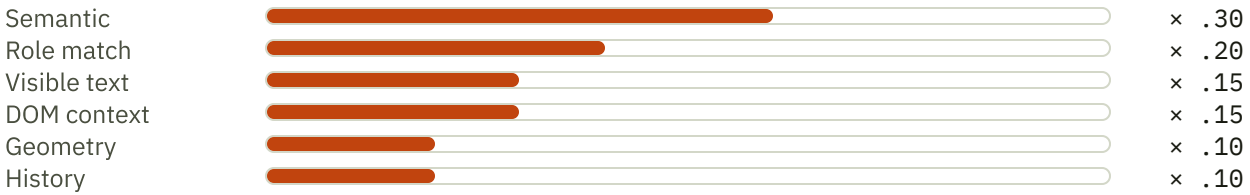
FLAKY
Evidence points to timing/non-determinism, not a real change.

ENVIRONMENT
The target itself is unreachable or degraded — not a test problem at all.

UNKNOWN
No row matched confidently. Escalated rather than guessed.

Heal — six signals produce one score

A candidate replacement is scored on six independent signals, combined by weight, then capped at a ceiling that depends on how it was found — the ceiling is a ceiling, never a bonus.



`score = min(raw, baseTrust[strategy])` — a locator found by role+name can reach 1.00; one found only by XPath is capped at 0.20, however well it otherwise scores.

ONLY ONE CANDIDATE SHAPE IS ELIGIBLE

Resolved count	**must equal exactly 1**
Filtered	**before scoring, not after**
Ranked pool	**top 5, no duplicates**

A locator that resolves to zero or to many elements never reaches the scorer at all — ambiguity is a filter, not a penalty.

Heal — five vetoes decide before the score does

Vetoes run first. Any one of them fires and the score is never even consulted — a veto is a claim about a category, not a very low number.

V1
Assertion target

A failed truth-claim step is never eligible for a patch.

V2
Destructive verb

Never heal a safe control into "delete", "cancel", "refund"...

V3
Numeric drift

Expected vs. actual differ only in a number or currency.

V4
Ambiguity

Top two candidates within 0.05 of each other.

V5
Runtime regression

A new console error or 5xx appeared since baseline.

`Veto fired` → BLOCKED, unconditionally | `score ≥ 0.85, margin > 0.05` → AUTO_HEAL | `score ≥ 0.65` → ESCALATE_FOR_REVIEW

Why vetoes, not just a higher confidence threshold?

A single scalar threshold cannot express "no score is high enough here" — and two real failure shapes need exactly that sentence.

CRITERION	A — THRESHOLD ONLY	B — VETOES FIRST (CHOSEN)
Catches *"Place order"* → *"Delete order"*	No — scores ≈0.71, high on every signal	**Yes — V2 fires, score never consulted**
Catches ₹999 → ₹9,999	No — edit distance 1, scores ≈0.99	**Yes — V3 fires**
Cost of buying more safety	Also blocks legitimate heals	**Blocks only the named class**
Explainable on stage	"0.91 ≥ 0.85"	**"0.71 said heal; V2 said no — veto wins"**
Needs a labelled corpus to defend	Yes — a threshold is a claim about a distribution	**No — a veto is a claim about a category**

The deciding argument is cost asymmetry: a false block costs one engineer a minute against a full evidence pack; a false heal costs a production incident *and* the credibility of every green result the suite has ever produced.

Why can't the model's judgment ever block the pipeline?

The obvious worry is a model blocking too much. The dangerous case is the opposite one: a model clearing a blocker nobody should have cleared.

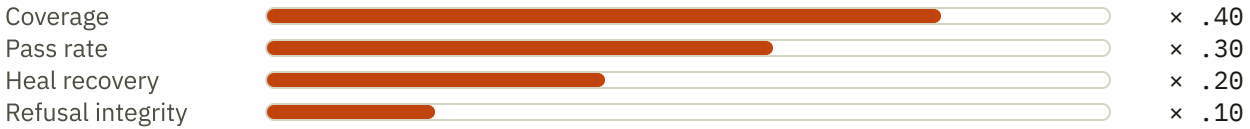
CRITERION	A — MODEL GAPS CAN BLOCK	B — ONLY ARITHMETIC BLOCKS (CHOSEN)
Same plan, same verdict, five times	No	**Yes**
Behaviour with no API key	The gate silently weakens	**Identical**
Can a persuasive model stall a lap forever?	Yes	**No**
Can a persuasive model wave a bad plan through?	Yes	**No — it can't clear a blocker either**
Explains in one sentence	"The model decides"	**"Arithmetic decides; the model advises — both are recorded"**

Model-authored gaps are clamped to `MAJOR` in code before merge. A genuinely critical semantic gap still reaches the report — it just can't silently stop, or silently pass, the lap on its own authority.

Report — every number, recomputable by hand

REPORT

The robustness score is a weighted sum of four components, every one of them a plain ratio over stored results — nothing in it is a claim to take on faith.



ALSO IN THE REPORT

Critic rounds	**every score, every verdict**
Healer actions	**healed / blocked / escalated, with evidence**
Residual gaps	**named explicitly, never hidden in the score**
Delight Score	**alongside, never merged in**

Watch it happen — `forge ui`

The same pipeline, streamed live over server-sent events, to a dashboard built for someone who has never seen a test suite before.

- **One input: a URL, and credentials if the app needs them.**
  Nothing to configure — press Run and every stage above starts streaming.
- **Plain-English narration, technical detail on demand.**
  Every timeline card leads with what's happening and why, in plain language — the raw score, locator, or veto id is one click away, never deleted.
- **Live screenshots, not a log file.**
  Every page FORGE visits streams to the screen as it's captured — the same evidence the Design Critic is scoring, visible in real time.
- **A continuous progress indicator.**
  The dashboard always shows which of the eight stages is currently running, through to "Generating your report" — never a silent gap.

Where this sits against the alternatives

APPROACH	SCALES WITH RELEASES	TELLS "MOVED" FROM "BROKEN"	AUDITABLE WITHOUT TRUSTING A MODEL
Manual QA	No	**Yes, if the person notices**	Yes, but nothing is recorded
Scripted E2E (as-is)	Yes	No — every drift is just a failure	Yes, but brittle
Similarity-only auto-heal	Yes	**No — heals through real bugs**	No — one scalar, opaque
FORGE	**Yes**	**Yes — six-cause triage before any repair**	**Yes — arithmetic holds with the model off**

The differentiator isn't "it uses AI." It's that the parts which must never be probabilistic — the floor, the vetoes, the blocker gate — aren't. The model's judgment shows up everywhere it adds value, and nowhere it could quietly cost trust.

Built, demo-scoped, and deferred — named honestly

The same discipline the pipeline applies to a web app's coverage, applied to this project's own status: gaps are named, never hidden inside a green checkmark.

Ph0–Ph2

Built & tested

Workspace guardrails, schemas/FSM/store, perception & the Explorer — confirmed live against real targets.

Ph3–Ph6

Demo fast-track

One real, deterministic-only vertical slice shipped end to end — narrower than the full spec (e.g. one locator rung, four-term coverage score), honestly labelled as such in-repo.

Ph5 (Triage/Heal)

Real pass shipped

The full ten-row triage table, six-signal scorer, and all five vetoes — 111 tests green — superseding the earlier fast-track stub.

Ph7 (Design Critic)

Built & tested

Ten checks shipped end to end; seven more (DC-01/02/05/07–10) and masked pixel diffing deliberately deferred, not attempted.

Arithmetic decides. The model advises. Both are on the record.

That one sentence is the whole design — applied to the Critic's floor, the Healer's vetoes, and every verdict in between. Nothing in this pipeline asks you to trust a claim it can't also show you the evidence for.

```
$ pnpm forge ui
```